

فصل ۳

گزینه بلند	گزینه کوتاه	توضیح
--all	-a	نمایش همه فایل‌ها حتی فایل‌های مخفی شروع شده با .
--almost-all	-A	مشابه a- اما بدون نمایش . و ..
--directory	-d	نمایش خود دایرکتوری به جای محتویات آن
--classify	-F	افزودن نماد تشخیص نوع فایل مثلاً / برای پوشه‌ها
--human-readable	-h	نمایش اندازه‌ها به صورت قابل خواندن (GB, MB, KB)
—	-l	فرمت طولانی
--reverse	-r	ترتیب معکوس در نمایش
—	-S	مرتب‌سازی بر اساس اندازه
—	-t	مرتب‌سازی بر اساس زمان آخرین ویرایش

مسیر	توضیح
/	ریشه‌ی کل سیستم فایل
/bin	برنامه‌های ضروری برای بوت و اجرای سیستم
/boot	شامل کرنل لینوکس و فایل‌های راه‌اندازی
/dev	شامل دستگاه‌های سخت‌افزاری (device nodes)
/etc	فایل‌های پیکربندی سیستمی (متنی)
/home	پوشه‌ی شخصی هر کاربر
/lib	کتابخانه‌های مشترک سیستم
/lost+found	برای بازیابی داده‌های از دست رفته پس از خرابی فایل سیستم

مسیر	توضیح
/media	محل اتصال خودکار دستگاه‌های جانبی
/mnt	برای اتصال دستی رسانه‌ها (در سیستم‌های قدیمی‌تر)
/opt	نرم‌افزارهای اختیاری یا تجاری
/proc	فایل سیستم مجازی شامل اطلاعات زنده از کرنل
/root	دایرکتوری کاربر root
/sbin	ابزارهای سیستمی برای مدیر سیستم
/tmp	فایل‌های موقت
/usr	بزرگ‌ترین بخش، شامل برنامه‌ها و فایل‌های کاربری
/usr/share/doc	مستندات نرم‌افزارها
/var	فایل‌هایی که به مرور تغییر می‌کنند (مانند لاگ‌ها و ایمیل‌ها)

تمرین پیشنهادی:

(۱) وارد مسیر `etc/` شوید و فایل `fstab` را با دستور `less` باز کنید.

(۲) فایل `passwd` را بررسی کنید تا ساختار حساب‌ها را ببینید.

تمرین‌های پایانی فصل ۳

(۱) تفاوت بین `file`, `ls` و `less` را توضیح دهید.

(۲) با استفاده از `ls -lh /bin` فایل‌های بزرگ‌تر از **1 MB** را شناسایی کنید.

(۳) با دستور `file` نوع چند فایل در `/etc` را بررسی کنید.

(۴) برای مرور فایل لاگ اصلی سیستم از **less /var/log/syslog** استفاده کنید.

(۵) لینک نمادینی از **/etc/passwd** به نام **userlist** بسازید (در پوشه خانگی خود).

کاراکتر	مفهوم
*	هر تعداد کاراکتر
?	یک کاراکتر منفرد
[abc]	هر کاراکتری از بین مجموعه a ، b ، c
[!abc]	هر کاراکتری به جز a ، b یا c
[[:class:]]	اعضای یک کلاس خاص (مثلاً حروف کوچک، ارقام و...)

inode در لینوکس و سیستم‌عامل‌های شبیه Unix یک ساختار داده‌ای در دیسک است که اطلاعات مربوط به یک فایل یا پوشه را نگه می‌دارد، اما خود نام فایل را ذخیره نمی‌کند.

مثال) فرض کنید فایل‌هایی با نام **file1.txt** و **file2.txt** وجود دارند. ساختار روی دیسک ممکن است شبیه این باشد:

Directory (folder)

file1.txt	--> inode 100
file2.txt	--> inode 101

Inode Table

inode 100	--> location of data blocks for file1.txt
inode 101	--> location of data blocks for file2.txt

وقتی یک **hard link** با دستور **ln file1.txt link1.txt** ایجاد می‌کنید، نام جدیدی به همان **inode** اشاره می‌کند:

```
link1.txt --> inode 100
file1.txt --> inode 100
```

پس هر دو نام فایل به یک محتوای واقعی در دیسک اشاره می‌کنند.

hard link

در لینوکس، **hard link** یکی از روش‌های قدیمی ایجاد لینک به فایل‌ها است که با **symbolic link** مدرن تفاوت دارد. هر فایل به‌طور پیش‌فرض یک **hard link** دارد که نام فایل را مشخص می‌کند و ایجاد یک **hard link** جدید در واقع یک ورودی اضافی در دایرکتوری است که به همان فایل اشاره می‌کند، بدون اینکه محتوای فایل کپی شود. هر **hard link** به یک **inode** اشاره دارد؛ **inode** ساختار داده‌ای روی دیسک است که اطلاعات فایل مانند نوع، اندازه، مالک، مجوزها، زمان‌ها و مکان داده‌ها روی دیسک را ذخیره می‌کند، اما نام فایل را ذخیره نمی‌کند. محدودیت‌های **hard link** این است که نمی‌تواند به فایل خارج از همان فایل سیستم یا به پوشه اشاره کند. نکته مهم این است که هیچ "فایل اصلی" واقعی وجود ندارد؛ همه لینک‌ها به همان **inode** اشاره دارند. بنابراین اگر یکی از لینک‌ها حذف شود، داده‌ها همچنان روی دیسک باقی می‌مانند و دیگر **hard link** ها می‌توانند به آن دسترسی داشته باشند، و تنها زمانی که تمام لینک‌ها حذف شوند، داده‌ها از دیسک پاک می‌شوند. به همین دلیل، **hard link** دقیقاً همان فایل است و غیرقابل تشخیص از آن می‌باشد، برخلاف **symbolic link** که یک مسیر به فایل دیگر است و حذف آن روی فایل اصلی تأثیری ندارد. این مفاهیم، به ویژه نقش **inode** و رفتار حذف لینک‌ها، برای درک نحوه مدیریت فایل‌ها و لینک‌ها در لینوکس اهمیت زیادی دارند.

symbolic link

در لینوکس، **symbolic link** یا **symlink** یک لینک مدرن است که به جای نگه داشتن داده‌های فایل، مسیر فایل یا پوشه دیگری را ذخیره می‌کند و مثل یک میانبر عمل می‌کند. بر خلاف **hard link** که فقط می‌تواند به فایل‌های همان فایل سیستم اشاره کند، **symlink** می‌تواند به هر فایل یا پوشه‌ای حتی در پارتیشن‌های دیگر اشاره کند و در لیست دایرکتوری با **ls -l** و علامت **->** دیده می‌شود. اگر فایل مقصد حذف شود، **symlink** خراب می‌شود و دیگر کار نمی‌کند. **hard link** چند نام برای یک فایل ایجاد می‌کند و با فایل اصلی تفاوتی ندارد؛ داده‌ها تا زمانی که همه لینک‌ها حذف شوند، روی دیسک باقی می‌مانند، ولی محدود به همان فایل سیستم هستند و نمی‌توانند به پوشه اشاره کنند. به همین دلیل، **hard link** برای ایجاد چند نام برای یک فایل و صرفه‌جویی در فضا کاربرد دارد، اما **symlink** برای ایجاد میانبر و دسترسی راحت به فایل‌ها و پوشه‌ها در مسیرهای مختلف مناسب‌تر است.

تمرین پیشنهادی فصل ۴ برای دانشجویان:

- (۱) در دایرکتوری خانگی خود یک پوشه **test** بسازید و در آن فایل‌های مختلف با پسوندهای **.txt**, **.dat**, **.sh** ایجاد کنید.
- (۲) با **wildcard** فقط فایل‌های **txt** را به پوشه جدید کپی کنید.
- (۳) یکی از فایل‌ها را به صورت **hard link** و دیگری را به صورت **symbolic link** لینک کنید.
- (۴) بررسی کنید در خروجی **ls -li** چه تفاوتی در **inode** بین لینک‌ها وجود دارد.
- (۵) در پایان با **rm -r** محیط خود را تمیز کنید.